Customer Shopping Behavior Analysis

1. **Project Overview**

This project analyzes customer shopping behavior using transactional data from 3,900 purchases across various product categories. The goal is to uncover insights into spending patterns, customer segments, product preferences, and subscription behavior to guide strategic business decisions.

2. **Dataset Summary**

- Rows: 3,900
- Columns: 18
- Key Features:
  - Customer demographics (Age, Gender, Location, Subscription Status)
  - Purchase details (Item Purchased, Category, Purchase Amount, Season, Size, Color)
  - Shopping behavior (Discount Applied, Promo Code Used, Previous Purchases, Frequency of Purchases, Review Rating, Shipping Type)
- Missing Data: 37 values in Review Rating column

3. **Exploratory Data Analysis using Python**

We began with data preparation and cleaning in Python:

- **Data Loading:** Imported the dataset using pandas.

- **Initial Exploration:** Used df.info()to check structure and .describe()for summary statistics.

```python
import pandas as pd
```
[3] Python

```python
df=pd.read_csv("C:/Users/PCSS/OneDrive/Desktop/PROJECT/customer_shopping_behavior.csv")
```
[4] Python

```python
df.head()
```
[5] Python

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Payment Method	Frequency of Purchases
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Express	Yes	Yes	14	Venmo	Fortnightly
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Express	Yes	Yes	2	Cash	Fortnightly
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Free Shipping	Yes	Yes	23	Credit Card	Weekly
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	Next Day Air	Yes	Yes	49	PayPal	Weekly
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Free Shipping	Yes	Yes	31	PayPal	Annually

```python
df.info()
```
[6] Python

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Customer ID             3900 non-null   int64
 1   Age                     3900 non-null   int64
 2   Gender                  3900 non-null   object
 3   Item Purchased          3900 non-null   object
 4   Category                3900 non-null   object
 5   Purchase Amount (USD)   3900 non-null   int64
 6   Location                3900 non-null   object
 7   Size                    3900 non-null   object
 8   Color                   3900 non-null   object
 9   Season                  3900 non-null   object
 10  Review Rating           3863 non-null   float64
 11  Subscription Status     3900 non-null   object
 12  Shipping Type           3900 non-null   object
 13  Discount Applied        3900 non-null   object
 14  Promo Code Used         3900 non-null   object
 15  Previous Purchases      3900 non-null   int64
 16  Payment Method          3900 non-null   object
 17  Frequency of Purchases  3900 non-null   object
```

```python
df.describe()
df.describe(include='all')
```

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Payment Method	Frequency of Purchases
count	3900.000000	3900.000000	3900	3900	3900	3900.000000	3900	3900	3900	3900	3863.000000	3900	3900	3900	3900	3900.000000	3900	3900
unique	NaN	NaN	2	25	4	NaN	50	4	25	4	NaN	2	6	2	2	NaN	6	7
top	NaN	NaN	Male	Blouse	Clothing	NaN	Montana	M	Olive	Spring	NaN	No	Free Shipping	No	No	NaN	PayPal	Every 3 Months
freq	NaN	NaN	2652	171	1737	NaN	96	1755	177	999	NaN	2847	675	2223	2223	NaN	677	584
mean	1950.500000	44.068462	NaN	NaN	NaN	59.764359	NaN	NaN	NaN	NaN	3.750065	NaN	NaN	NaN	NaN	25.351538	NaN	NaN
std	1125.977353	15.207589	NaN	NaN	NaN	23.685392	NaN	NaN	NaN	NaN	0.716983	NaN	NaN	NaN	NaN	14.447125	NaN	NaN
min	1.000000	18.000000	NaN	NaN	NaN	20.000000	NaN	NaN	NaN	NaN	2.500000	NaN	NaN	NaN	NaN	1.000000	NaN	NaN
25%	975.750000	31.000000	NaN	NaN	NaN	39.000000	NaN	NaN	NaN	NaN	3.100000	NaN	NaN	NaN	NaN	13.000000	NaN	NaN
50%	1950.500000	44.000000	NaN	NaN	NaN	60.000000	NaN	NaN	NaN	NaN	3.800000	NaN	NaN	NaN	NaN	25.000000	NaN	NaN
75%	2925.250000	57.000000	NaN	NaN	NaN	81.000000	NaN	NaN	NaN	NaN	4.400000	NaN	NaN	NaN	NaN	38.000000	NaN	NaN
max	3900.000000	70.000000	NaN	NaN	NaN	100.000000	NaN	NaN	NaN	NaN	5.000000	NaN	NaN	NaN	NaN	50.000000	NaN	NaN

- **Missing Data Handling:** Checked for null values and imputed missing values in the Review Ratingcolumn using the median rating of each product category.

```python
df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(lambda x: x.fillna(x.median()))
```

```python
df.isnull().sum()
```

```
Customer ID               0
Age                       0
Gender                    0
Item Purchased            0
Category                  0
Purchase Amount (USD)     0
Location                  0
Size                      0
Color                     0
Season                    0
Review Rating             0
Subscription Status       0
Shipping Type             0
Discount Applied          0
Promo Code Used           0
Previous Purchases        0
Payment Method            0
Frequency of Purchases    0
dtype: int64
```

- **Column Standardization:** Renamed columns to **snake case** for better readability and documentation.

```python
df.columns = df.columns.str.lower()
df.columns=df.columns.str.replace(' ','_')
df.rename(columns={'Purchase_amount_(usd)':'purchase_amount'})
```

	customer_id	age	gender	item_purchased	category	purchase_amount_(usd)	location	size	color	season	review_rating	subscription_status	shipping_type	discount_applied	promo_code_used	previous_purchases	payment_method	frequency_of_purchases
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Express	Yes	Yes	14	Venmo	Fortnightly
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Express	Yes	Yes	2	Cash	Fortnightly
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Free Shipping	Yes	Yes	23	Credit Card	Weekly
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	Next Day Air	Yes	Yes	49	PayPal	Weekly
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Free Shipping	Yes	Yes	31	PayPal	Annually
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
3895	3896	40	Female	Hoodie	Clothing	28	Virginia	L	Turquoise	Summer	4.2	No	2-Day Shipping	No	No	32	Venmo	Weekly
3896	3897	52	Female	Backpack	Accessories	49	Iowa	L	White	Spring	4.5	No	Store Pickup	No	No	41	Bank Transfer	Bi-Weekly
3897	3898	46	Female	Belt	Accessories	33	New Jersey	L	Green	Spring	2.9	No	Standard	No	No	24	Venmo	Quarterly
3898	3899	44	Female	Shoes	Footwear	77	Minnesota	S	Brown	Summer	3.8	No	Express	No	No	24	Venmo	Weekly
3899	3900	52	Female	Handbag	Accessories	81	California	M	Beige	Spring	3.1	No	Store Pickup	No	No	33	Venmo	Quarterly

3900 rows × 18 columns

- **Feature Engineering:**
  - Created **age_group** column by binning customer ages.
  - Created **purchase_frequency_days** column from purchase data.

```python
# create a new column age_group
labels = ['Young Adult', 'Adult', 'Middle-aged', 'Senior']
df['age_group'] = pd.qcut(df['age'], q=4, labels = labels)
```

```python
df[['age','age_group']].head(10)
```

	age	age_group
0	55	Middle-aged
1	19	Young Adult
2	50	Middle-aged
3	21	Young Adult
4	45	Middle-aged
5	46	Middle-aged
6	63	Senior
7	27	Young Adult
8	26	Young Adult
9	57	Middle-aged

```python
# create new column purchase_frequency_days

frequency_mapping = {
    'Fortnightly': 14,
    'Weekly': 7,
    'Monthly': 30,
    'Quarterly': 90,
    'Bi-Weekly': 14,
    'Annually': 365,
    'Every 3 Months': 90
}

df['purchase_frequency_days'] = df['frequency_of_purchases'].map(frequency_mapping)
df[['purchase_frequency_days','frequency_of_purchases']].head(10)
```

	purchase_frequency_days	frequency_of_purchases
0	14	Fortnightly
1	14	Fortnightly
2	7	Weekly

- **Data Consistency Check:** Verified if discount_appliedand promo_code_used were redundant; dropped promo_code_used.

```python
df[['discount_applied','discount_applied']].head(10)
```

	discount_applied	discount_applied
0	Yes	Yes
1	Yes	Yes
2	Yes	Yes
3	Yes	Yes
4	Yes	Yes
5	Yes	Yes
6	Yes	Yes
7	Yes	Yes
8	Yes	Yes
9	Yes	Yes

```python
# to check some columns have same values
(df['discount_applied']== df['promo_code_used']).all()
```

```
np.True_
```

```python
# Dropping promo code used column

df = df.drop('promo_code_used', axis=1)
```

```python
df.columns
```

```
Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount_(usd)', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'previous_purchases', 'payment_method',
       'frequency_of_purchases', 'age_group', 'purchase_frequency_days'],
      dtype='object')
```

- **Database Integration:** Connected Python script to Mysql and loaded the cleaned DataFrame into the database for SQL analysis.

Generate | + Code | + Markdown

```
pip install pymysql sqlalchemy
```

[25] Python

```
Requirement already satisfied: pymysql in c:\users\pcss\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (1.1.1)
Requirement already satisfied: sqlalchemy in c:\users\pcss\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (2.0.45)
Requirement already satisfied: greenlet>=1 in c:\users\pcss\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from sqlalchemy) (3.3.0)
Requirement already satisfied: typing-extensions>=4.6.0 in c:\users\pcss\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from sqlalchemy) (4.14.1)
Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 25.1.1 -> 25.3
[notice] To update, run: C:\Users\PCSS\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip
```

```python
from sqlalchemy import create_engine

# MySQL connection
username = "root"
password = "112233"
host = "127.0.0.1"
port = "3306"
database = "customer_behavior"

engine = create_engine(f"mysql+pymysql://{username}:{password}@{host}:{port}/{database}")

# Write DataFrame to MySQL
table_name = "customer"   # choose any table name
df.to_sql(table_name, engine, if_exists="replace", index=False)

# Read back sample
pd.read_sql("SELECT * FROM customer LIMIT 5;", engine)
```

[30] Python

	customer_id	age	gender	item_purchased	category	purchase_amount_(usd)	location	size	color	season	review_rating	subscription_status	shipping_type	discount_applied	previous_purchases	payment_method	frequency_of_purchases	age_group	purchase_frequency_days
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Express	Yes	14	Venmo	Fortnightly	Middle-aged	14
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Express	Yes	2	Cash	Fortnightly	Young Adult	14
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Free Shipping	Yes	23	Credit Card	Weekly	Middle-aged	7
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	Next Day Air	Yes	49	PayPal	Weekly	Young Adult	7
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Free Shipping	Yes	31	PayPal	Annually	Middle-aged	365

4. Data Analysis using SQL (Business Transactions)

We performed structured analysis in Mysql workbench to answer key business questions:

1. **Revenue by Gender** – Compared total revenue generated by male vs. female customers.

customer_behavior_sql_queries

Limit to 1000 rows

```sql
-- SELECT * FROM customer_behavior.customer_behavior;
USE customer_behavior;
select * from customer_behavior;

-- Q1. What is the total revenue generated by male vs. female customers?
USE customer_behavior;
SELECT gender,SUM(`purchase_amount_(usd)`) AS revenue
FROM customer_behavior
GROUP BY gender;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

gender	revenue
Male	157890
Female	75191

2. **High-Spending Discount Users** – Identified customers who used discounts but still spent above the average purchase amount.

customer_behavior_sql_queries

Limit to 1000 rows

```sql
-- Q2. Which customers used a discount but still spent more than the average purchase amount?
SELECT
    customer_id,
    `purchase_amount_(usd)`
FROM customer_behavior
WHERE discount_applied = 'Yes'
  AND `purchase_amount_(usd)` >= (
      SELECT AVG(`purchase_amount_(usd)`)
      FROM customer_behavior
    );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	purchase_amount_(usd)
2	64
3	73
4	90
7	85
9	97

customer_behavior 2

3. **Top 5 Products by Rating** – Found products with the highest average review ratings.

customer_behavior_sql_queries

Limit to 1000 rows

```sql
-- Q3. Which are the top 5 products with the highest average review rating?
SELECT
    item_purchased,
    ROUND(AVG(review_rating),2) AS avg_review_rating
FROM customer_behavior
GROUP BY item_purchased
ORDER BY avg_review_rating DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

item_purchased	avg_review_rating
Gloves	3.86
Sandals	3.84
Boots	3.82
Hat	3.8
Handbag	3.78

4. **Shipping Type Comparison** – Compared average purchase amounts between Standard and Express shipping.

customer_behavior_sql_queries

Limit to 1000 rows

```sql
-- Q4. Compare the average Purchase Amounts between Standard and Express Shipping.
SELECT
    shipping_type,
    ROUND(AVG(`purchase_amount_(usd)`), 2) AS avg_purchase_amount
FROM customer_behavior
WHERE shipping_type IN ('Standard', 'Express')
GROUP BY shipping_type
HAVING AVG(`purchase_amount_(usd)`) > 0;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

item_purchased	avg_review_rating
Gloves	3.86
Sandals	3.84
Boots	3.82
Hat	3.8
Handbag	3.78

5. **Subscribers vs. Non-Subscribers** – Compared average spend and total revenue across subscription status.

```sql
-- Q5. Do subscribed customers spend more? Compare average spend and total revenue between subscribers and non-subscribers
SELECT
    subscription_status,
    COUNT(customer_id) AS total_customers,
    ROUND(AVG(`purchase_amount_(usd)`), 2) AS avg_spend,
    ROUND(SUM(`purchase_amount_(usd)`), 2) AS total_revenue
FROM customer_behavior
GROUP BY subscription_status
ORDER BY total_revenue, avg_spend DESC;
```

subscription_status	total_customers	avg_spend	total_revenue
Yes	1053	59.49	62645
No	2847	59.87	170436

6. **Discount-Dependent Products** – Identified 5 products with the highest percentage of discounted purchases.

```sql
-- Q6. Which 5 products have the highest percentage of purchases with discounts applied?
SELECT item_purchased,
       ROUND(100.0 * SUM(CASE WHEN discount_applied = 'Yes' THEN 1 ELSE 0 END)/COUNT(*),2) AS discount_rate
FROM customer_behavior
GROUP BY item_purchased
ORDER BY discount_rate DESC
LIMIT 5;
```

item_purchased	discount_rate
Hat	50.00
Sneakers	49.66
Coat	49.07
Sweater	48.17
Pants	47.37

7. **Customer Segmentation** – Classified customers into New, Returning, and Loyal segments based on purchase history.

```sql
-- Q7.Segment customers into New, Returning, and Loyal based on their total number of previous purchases, and show the count
with customer_type as (
SELECT customer_id, previous_purchases,
CASE
    WHEN previous_purchases = 1 THEN 'New'WHEN previous_purchases
    BETWEEN 2 AND 10 THEN 'Returning'ELSE 'Loyal'END AS customer_segment
    FROM customer_behavior)
select customer_segment,count(*) AS "Number of Customers"
from customer_type
group by customer_segment;
```

customer_segment	Number of Customers
Loyal	3116
Returning	701
New	83

8. **Top 3 Products per Category** – Listed the most purchased products within each category.

```sql
-- Q8. What are the top 3 most purchased products within each category?
WITH item_counts AS (
    SELECT category,
           item_purchased,
           COUNT(customer_id) AS total_orders,
           ROW_NUMBER() OVER (PARTITION BY category ORDER BY COUNT(customer_id) DESC) AS item_rank FROM customer
    GROUP BY category, item_purchased)
SELECT item_rank,category, item_purchased, total_orders
FROM item_counts
WHERE item_rank <=3;
```

item_rank	category	item_purchased	total_orders
1	Accessories	Jewelry	171
2	Accessories	Sunglasses	161
3	Accessories	Belt	161
1	Clothing	Blouse	171
2	Clothing	Pants	171

9. **Repeat Buyers & Subscriptions** – Checked whether customers with >5 purchases are more likely to subscribe.

customer_behavior_sql_queries*

Limit to 1000 rows

```sql
-- Q9. Are customers who are repeat buyers (more than 5 previous purchases) also likely to subscribe?
SELECT subscription_status,
       COUNT(customer_id) AS repeat_buyers
FROM customer_behavior
WHERE previous_purchases > 5
GROUP BY subscription_status;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

subscription_status	repeat_buyers
Yes	958
No	2518

10. **Revenue by Age Group** – Calculated total revenue contribution of each age group.

customer_behavior_sql_queries*

Limit to 1000 rows

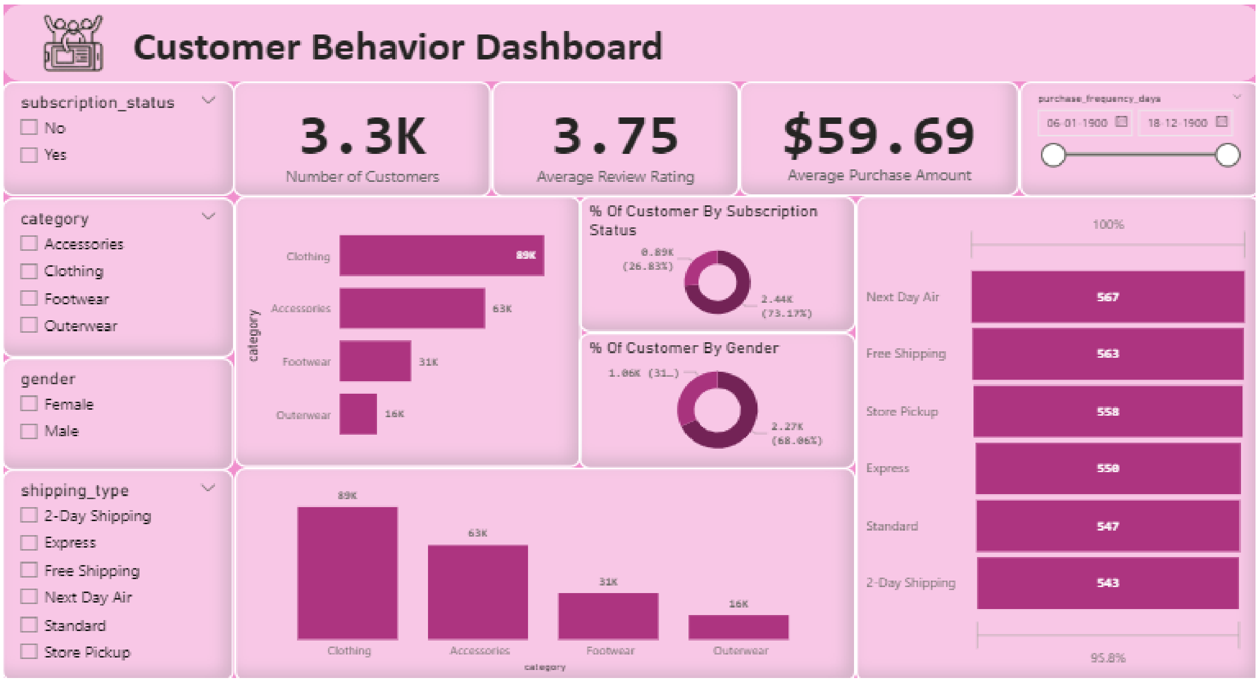
```sql
-- Q10. What is the revenue contribution of each age group?
SELECT
    age_group,
    SUM(`purchase_amount_(usd)`) AS total_revenue
FROM customer_behavior
GROUP BY age_group
ORDER BY total_revenue desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

age_group	total_revenue
Young Adult	62143
Middle-aged	59197
Adult	55978
Senior	55763

5. Dashboard in Power BI

Finally, we built an interactive dashboard in **Power BI** to present insights visually.



6. Business Recommendations

- **Boost Subscriptions** – Promote exclusive benefits for subscribers.
- **Customer Loyalty Programs** – Reward repeat buyers to move them into the "Loyal" segment.
- **Review Discount Policy** – Balance sales boosts with margin control.
- **Product Positioning** – Highlight top-rated and best-selling products in campaigns.
- **Targeted Marketing** – Focus efforts on high-revenue age groups and express-shipping users.